

COMANDOS ARTISAN PARA CREAR TESTS

(puedes dividirlos como quieras — sugerencia)

```
php artisan make:test AuthTest
php artisan make:test RoleMiddlewareTest
php artisan make:test PaymentServiceTest
php artisan make:test InscriptionServiceTest
php artisan make:test ClassesTest
php artisan make:test InscriptionsTest
```

TESTS CLASIFICADOS

1) AUTH TESTS

```
public function test_user_can_login()
{
    $user = User::factory()->create([
        'password' => bcrypt('pass123')
    ]);

    $this->post('/login', [
        'email' => $user->email,
        'password' => 'pass123'
    ]->assertStatus(302);
}

public function test_user_fails_with_bad_password()
{
    $user = User::factory()->create();

    $this->post('/login', [
        'email' => $user->email,
        'password' => 'wrong'
    ]->assertSessionHasErrors());
}
```

✓ 2) MIDDLEWARE TESTS

```
public function test_invalid_role_fails()
{
    $middleware = new RoleMiddleware();

    $this->expectException(\Exception::class);

    $middleware->handle(request(), function () {}, 'admin');
}
```

✓ 3) SERVICES TESTS

✓ PaymentServiceTest

```
public function test_correct_price_is_calculated()
{
    $service = new PaymentService();

    $result = $service->calculate(10, 3); // 10€/ud * 3

    $this->assertEquals(30, $result);
}
```

✓ InscriptionServiceTest

```
public function test_prevents_duplicate_inscriptions()
{
    $service = new InscriptionService();

    $client = User::factory()->create(['role' => 'client']);
    $class = Classes::factory()->create();

    $service->create($client->id, $class->id);

    $this->expectException(\Exception::class);

    $service->create($client->id, $class->id);
}
```

✓ 4) CLASSES TESTS (CRUD + permisos)

✓ Crear Clase

```
public function test_admin_can_create_class()
{
    $admin = User::factory()->create(['role'=>'admin']);

    $this->actingAs($admin)
        ->post('/admin/classes', [
            'name' => 'Yoga'
        ])
        ->assertStatus(302);

    $this->assertDatabaseHas('classes', ['name'=>'Yoga']);
}

public function test_client_cannot_create_class()
{
    $client = User::factory()->create(['role'=>'client']);

    $this->actingAs($client)
        ->post('/admin/classes', [
            'name' => 'Yoga'
        ])
        ->assertStatus(403);
}
```

✓ Read

```
public function test_client_can_view_classes()
{
    $client = User::factory()->create(['role'=>'client']);

    $this->actingAs($client)
        ->get('/classes')
        ->assertStatus(200);
}
```

(Duplicado en tu mensaje; lo mantengo solo 1 vez)

✓ Update

```
public function test_admin_can_update_class()
{
    $admin = User::factory()->create(['role'=>'admin']);
    $class = Classes::factory()->create();

    $this->actingAs($admin)
        ->put("/admin/classes/{ $class->id}", [
            'name' => 'Updated',
        ])
        ->assertStatus(302);

    $this->assertDatabaseHas('classes', ['name'=>'Updated']);
}

public function test_client_cannot_update_class()
{
    $client = User::factory()->create(['role'=>'client']);
    $class = Classes::factory()->create();

    $this->actingAs($client)
        ->put("/admin/classes/{ $class->id}", [
            'name' => 'Fake Update',
        ])
        ->assertStatus(403);
}
```

✓ Delete

```
public function test_admin_can_delete_class()
{
    $admin = User::factory()->create(['role'=>'admin']);
    $class = Classes::factory()->create();

    $this->actingAs($admin)
        ->delete("/admin/classes/{ $class->id}")
        ->assertStatus(302);

    $this->assertDatabaseMissing('classes', ['id'=>$class->id]);
}

public function test_client_cannot_delete_class()
{
    $client = User::factory()->create(['role'=>'client']);
    $class = Classes::factory()->create();

    $this->actingAs($client)
        ->delete("/admin/classes/{ $class->id}")
        ->assertStatus(403);
}
```

✓ Agrupador forbidden

```
/**
 * @dataProvider client_forbidden_routes
 */
public function test_client_cannot_manage_class($method, $uri)
{
    $client = User::factory()->create(['role'=>'client']);
    $class = Classes::factory()->create();

    $this->actingAs($client)
        ->json($method, str_replace('{id}', $class->id, $uri))
        ->assertStatus(403);
}

public function client_forbidden_routes()
{
    return [
        ['POST', '/admin/classes'],
        ['PUT', '/admin/classes/{id}'],
        ['DELETE', '/admin/classes/{id}'],
    ];
}
```

✓ Accesso

```
public function test_client_cannot_access_admin_routes()
{
    $client = User::factory()->create(['role'=>'client']);

    $this->actingAs($client)
        ->get('/admin/classes')
        ->assertStatus(403);
}
```



5) INSCRIPTIONS TESTS



Create

```
public function test_admin_can_create_inscription()
{
    $admin = User::factory()->create(['role'=>'admin']);
    $client = User::factory()->create(['role'=>'client']);
    $class = Classes::factory()->create();

    $this->actingAs($admin)
        ->post('/admin/inscriptions', [
            'client_id' => $client->id,
            'class_id' => $class->id,
        ])
        ->assertStatus(302);

    $this->assertDatabaseHas('inscriptions', [
        'client_id' => $client->id,
        'class_id' => $class->id
    ]);
}

public function test_client_can_create_own_inscription()
{
    $client = User::factory()->create(['role'=>'client']);
    $class = Classes::factory()->create();

    $this->actingAs($client)
        ->post('/clients/inscriptions', [
            'class_id' => $class->id,
        ])
        ->assertStatus(302);

    $this->assertDatabaseHas('inscriptions', [
        'client_id' => $client->id,
        'class_id' => $class->id
    ]);
}

public function test_client_cannot_inscribe_other_client()
{
    $client1 = User::factory()->create(['role'=>'client']);
    $client2 = User::factory()->create(['role'=>'client']);
    $class = Classes::factory()->create();

    $this->actingAs($client1)
        ->post('/clients/inscriptions', [
            'client_id'=>$client2->id,
            'class_id'=>$class->id
        ])
        ->assertStatus(403);
}
```

✓ Read

```
public function test_admin_can_view_any_inscription()
{
    $admin = User::factory()->create(['role'=>'admin']);
    $inscription = Inscription::factory()->create();

    $this->actingAs($admin)
        ->get("/admin/inscriptions/{$inscription->id}")
        ->assertStatus(200);
}

public function test_client_can_view_own_inscription()
{
    $client = User::factory()->create(['role'=>'client']);
    $inscription = Inscription::factory()->create([
        'client_id' => $client->id
    ]);

    $this->actingAs($client)
        ->get("/clients/inscriptions/{$inscription->id}")
        ->assertStatus(200);
}

public function test_client_cannot_view_other_inscriptions()
{
    $client1 = User::factory()->create(['role'=>'client']);
    $client2 = User::factory()->create(['role'=>'client']);

    $inscription = Inscription::factory()->create([
        'client_id' => $client2->id
    ]);

    $this->actingAs($client1)
        ->get("/clients/inscriptions/{$inscription->id}")
        ->assertStatus(403);
}
```

✓ Update

```
public function test_admin_can_update_inscription()
{
    $admin = User::factory()->create(['role'=>'admin']);
    $inscription = Inscription::factory()->create();

    $this->actingAs($admin)
        ->put("/admin/inscriptions/{$inscription->id}", [
            'notes' => 'updated'
        ])
        ->assertStatus(302);

    $this->assertDatabaseHas('inscriptions', [
        'id' => $inscription->id,
        'notes' => 'updated'
    ]);
}

public function test_client_cannot_update_inscription()
{
    $client = User::factory()->create(['role'=>'client']);
    $inscription = Inscription::factory()->create();

    $this->actingAs($client)
        ->put("/admin/inscriptions/{$inscription->id}", [
            'notes' => 'X'
        ])
        ->assertStatus(403);
}
```

✓ Delete

```
public function test_admin_can_delete_inscription()
{
    $admin = User::factory()->create(['role'=>'admin']);
    $inscription = Inscription::factory()->create();

    $this->actingAs($admin)
        ->delete("/admin/inscriptions/{$inscription->id}")
        ->assertStatus(302);

    $this->assertDatabaseMissing('inscriptions', [
        'id' => $inscription->id
    ]);
}

public function test_client_can_delete_own_inscription()
{
    $client = User::factory()->create(['role'=>'client']);
    $inscription = Inscription::factory()->create([
        'client_id' => $client->id
    ]);

    $this->actingAs($client)
        ->delete("/clients/inscriptions/{$inscription->id}")
        ->assertStatus(302);

    $this->assertDatabaseMissing('inscriptions', [
        'id' => $inscription->id
    ]);
}

public function test_client_cannot_delete_other_inscription()
{
    $client1 = User::factory()->create(['role'=>'client']);
    $client2 = User::factory()->create(['role'=>'client']);

    $inscription = Inscription::factory()->create([
        'client_id' => $client2->id
    ]);

    $this->actingAs($client1)
        ->delete("/clients/inscriptions/{$inscription->id}")
        ->assertStatus(403);
}
```

RESUMEN

Categoría	# tests
Auth	2
Middleware	1
Services	2
Classes	9
Inscriptions	12
TOTAL	 26

-  Estructura consistente → (admin CRUD + client restricted)
 -  Nivel crítico → seguridad, integridad de datos
-

A) PAYMENTS

Suelen estar estrictamente protegidos → solo admin

Admin puede crear payment

```
public function test_admin_can_create_payment()
{
    $admin = User::factory()->create(['role'=>'admin']);
    $inscription = Inscription::factory()->create();

    $this->actingAs($admin)
        ->post('/admin/payments', [
            'inscription_id' => $inscription->id,
            'amount' => 40
        ])
        ->assertStatus(302);

    $this->assertDatabaseHas('payments', [
        'inscription_id' => $inscription->id,
        'amount' => 40
    ]);
}
```

Client NO puede crear payment

```
public function test_client_cannot_create_payment()
{
    $client = User::factory()->create(['role'=>'client']);

    $this->actingAs($client)
        ->post('/admin/payments', [])
        ->assertStatus(403);
}
```

✓ Admin puede actualizar payment

```
public function test_admin_can_update_payment()
{
    $admin = User::factory()->create(['role'=>'admin']);
    $payment = Payment::factory()->create();

    $this->actingAs($admin)
        ->put("/admin/payments/{$payment->id}", [
            'amount' => 99
        ])
        ->assertStatus(302);

    $this->assertDatabaseHas('payments', [
        'id' => $payment->id,
        'amount' => 99
    ]);
}
```

✓ Client NO puede actualizar payment

```
public function test_client_cannot_update_payment()
{
    $client = User::factory()->create(['role'=>'client']);
    $payment = Payment::factory()->create();

    $this->actingAs($client)
        ->put("/admin/payments/{$payment->id}", [
            'amount' => 99
        ])
        ->assertStatus(403);
}
```

✓ Admin puede borrar payment

```
public function test_admin_can_delete_payment()
{
    $admin = User::factory()->create(['role'=>'admin']);
    $payment = Payment::factory()->create();

    $this->actingAs($admin)
        ->delete("/admin/payments/{$payment->id}")
        ->assertStatus(302);

    $this->assertDatabaseMissing('payments', ['id'=>$payment->id]);
}
```

✓ Client NO puede borrar payment

```
public function test_client_cannot_delete_payment()
{
    $client = User::factory()->create(['role'=>'client']);
    $payment = Payment::factory()->create();

    $this->actingAs($client)
        ->delete("/admin/payments/{$payment->id}")
        ->assertStatus(403);
}
```

✓ B) CLIENTS ✓

Los clients normalmente pueden **editar su propio perfil**, pero NO gestionar otros.

✓ Client puede actualizar su perfil

```
public function test_client_can_update_own_profile()
{
    $client = User::factory()->create(['role'=>'client']);

    $this->actingAs($client)
        ->put("/clients/{$client->id}", [
            'name' => 'Updated Name'
        ])
        ->assertStatus(302);

    $this->assertDatabaseHas('users', [
        'id' => $client->id,
        'name' => 'Updated Name'
    ]);
}
```

✓ Client NO puede actualizar otro perfil

```
public function test_client_cannot_update_other_client()
{
    $client1 = User::factory()->create(['role'=>'client']);
    $client2 = User::factory()->create(['role'=>'client']);

    $this->actingAs($client1)
        ->put("/clients/{$client2->id}", [])
        ->assertStatus(403);
}
```

✓ Admin puede actualizar cualquier client

```
public function test_admin_can_update_any_client()
{
    $admin = User::factory()->create(['role'=>'admin']);
    $client = User::factory()->create(['role'=>'client']);

    $this->actingAs($admin)
        ->put("/admin/clients/{$client->id}", [
            'name' => 'Updated'
        ])
        ->assertStatus(302);

    $this->assertDatabaseHas('users', [
        'id'=>$client->id,
        'name'=>'Updated'
    ]);
}
```

✓ Admin puede borrar client

```
public function test_admin_can_delete_client()
{
    $admin = User::factory()->create(['role'=>'admin']);
    $client = User::factory()->create(['role'=>'client']);

    $this->actingAs($admin)
        ->delete("/admin/clients/{$client->id}")
        ->assertStatus(302);

    $this->assertDatabaseMissing('users', ['id'=>$client->id]);
}
```

✓ C) USERS ✓

Realmente solapa con clients, pero si son entidades separadas:
admin puede CRUD, client no.

✓ Admin puede crear users

```
public function test_admin_can_create_user()
{
    $admin = User::factory()->create(['role'=>'admin']);

    $this->actingAs($admin)
        ->post('/admin/users', [
            'name' => 'Test',
            'email' => 'test@example.com',
            'password' => 'pass123',
            'role' => 'client',
        ])
        ->assertStatus(302);

    $this->assertDatabaseHas('users', [
        'email' => 'test@example.com'
    ]);
}
```

✓ Client NO puede crear users

```
public function test_client_cannot_create_user()
{
    $client = User::factory()->create(['role'=>'client']);

    $this->actingAs($client)
        ->post('/admin/users', [])
        ->assertStatus(403);
}
```

Update / Delete serían análogos al bloque de CLIENTS.

✓ D) COACHES ✓

Similar a CLASSES + USERS

✓ Admin puede crear coach

```
public function test_admin_can_create_coach()
{
    $admin = User::factory()->create(['role'=>'admin']);

    $this->actingAs($admin)
        ->post('/admin/coaches', [
            'name' => 'Coach A'
        ])
        ->assertStatus(302);

    $this->assertDatabaseHas('coaches', ['name'=>'Coach A']);
}
```

✓ Client NO puede crear coach

```
public function test_client_cannot_create_coach()
{
    $client = User::factory()->create(['role'=>'client']);

    $this->actingAs($client)
        ->post('/admin/coaches', [
            'name' => 'Coach X'
        ])
        ->assertStatus(403);
}
```

✓ Admin puede actualizar

```
public function test_admin_can_update_coach()
{
    $admin = User::factory()->create(['role'=>'admin']);
    $coach = Coach::factory()->create();

    $this->actingAs($admin)
        ->put("/admin/coaches/{$coach->id}", [
            'name' => 'Updated Coach'
        ])
        ->assertStatus(302);

    $this->assertDatabaseHas('coaches', [
        'id'=>$coach->id,
        'name'=>'Updated Coach'
    ]);
}
```

✓ Admin puede borrar

```
public function test_admin_can_delete_coach()
{
    $admin = User::factory()->create(['role'=>'admin']);
    $coach = Coach::factory()->create();

    $this->actingAs($admin)
        ->delete("/admin/coaches/{ $coach->id}")
        ->assertStatus(302);

    $this->assertDatabaseMissing('coaches', [
        'id'=>$coach->id
    ]);
}
```

✓ Client NO puede borrar

```
public function test_client_cannot_delete_coach()
{
    $client = User::factory()->create(['role'=>'client']);
    $coach = Coach::factory()->create();

    $this->actingAs($client)
        ->delete("/admin/coaches/{ $coach->id}")
        ->assertStatus(403);
}
```

✓ RESUMEN DE NUEVOS TESTS

✓ PAYMENTS

- ✓ admin create
- ✓ admin update
- ✓ admin delete
- ✓ client forbidden create
- ✓ client forbidden update
- ✓ client forbidden delete

✓ CLIENTS

- ✓ client update self
- ✓ client forbidden update others
- ✓ admin update client
- ✓ admin delete client

✓ **USERS**

- ✓ admin create
 - ✓ client forbidden create
- (+ opcional update/delete similares)

✓ **COACHES**

- ✓ admin create
- ✓ client forbidden create
- ✓ admin update
- ✓ admin delete
- ✓ client forbidden delete

Ahora sí → cobertura CRUD mínima razonable para todas las tablas principales.